
Artificial Intelligence

Tut/Lab week 6: Automated Planning

COSC2129/3118

Outline

- Assignment
- Highlights:
 - Search,
 - Knowledge representation & Inference
 - Planning
 - Planning algorithms
 - Exercises
- Q&A

Assignment 1

- Part 1:
 - Most of students worked very hard and submitted good solutions (code/algorithms & scores)
 - Score > 25, even Q9 (not required)
 - Some of the questions are not properly implemented, or with low or 0 score. You still get some points if you have the code implemented but with low score.
 - Code & comments should be improved.

❖ Congratulations!

- Part 2: Question?

Assessment 2

- In-class test
 - Open book with materials provided in the course
 - No AI tools, no internet search
- Cover all topics from week 4 to week 10
 - Multichoice
 - Short answer questions
- Test sheet template: week 9 (1 week before the test)
 - Not for solving/ finding solutions

Knowledge Representation & Reasoning

- Search
- Knowledge base
- Inference/ Reasoning

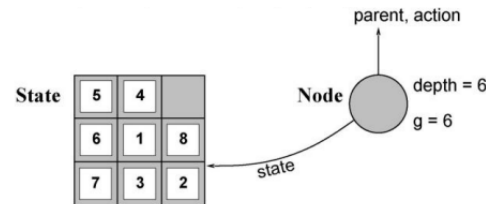
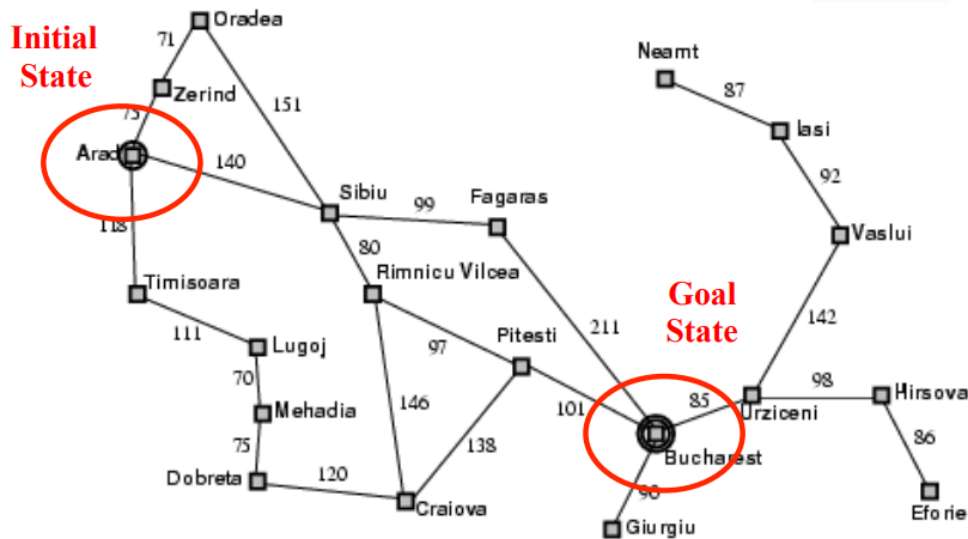
Automated planning

Materials:

- Book: [Artificial Intelligence: A Modern Approach](#)
 - Chapter 11.1-11.3 [11.3.2, 11.4-11.7]
- Other sources.

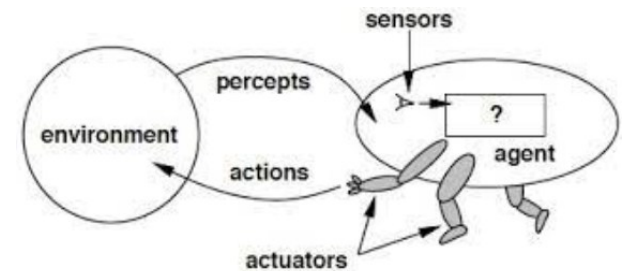
Search and planning

- Search



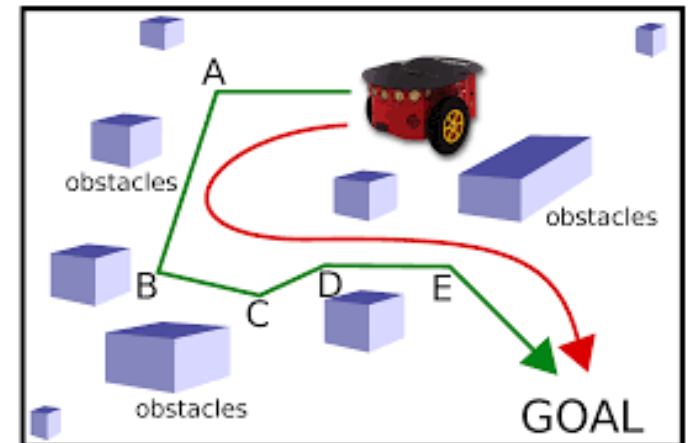
Intelligent Agent

KR & R (PL, FOL)



- Problem solving by search
 - BFS, DFS, DLS
 - Greedy BFS, A*

- Search and KR& R applied to planning?



Activity 1

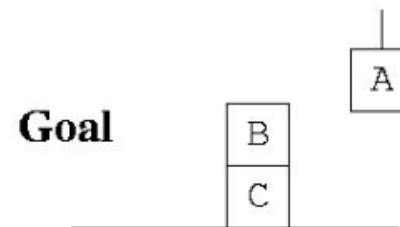
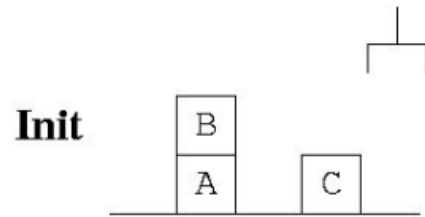
- Use AI tools (ChatGPT, Gemini, Grok) Search for 3 applications of automated planning in modern AI.
- What are they?
- What are states? state space? actions? goal?
- How to represent?

Quizz

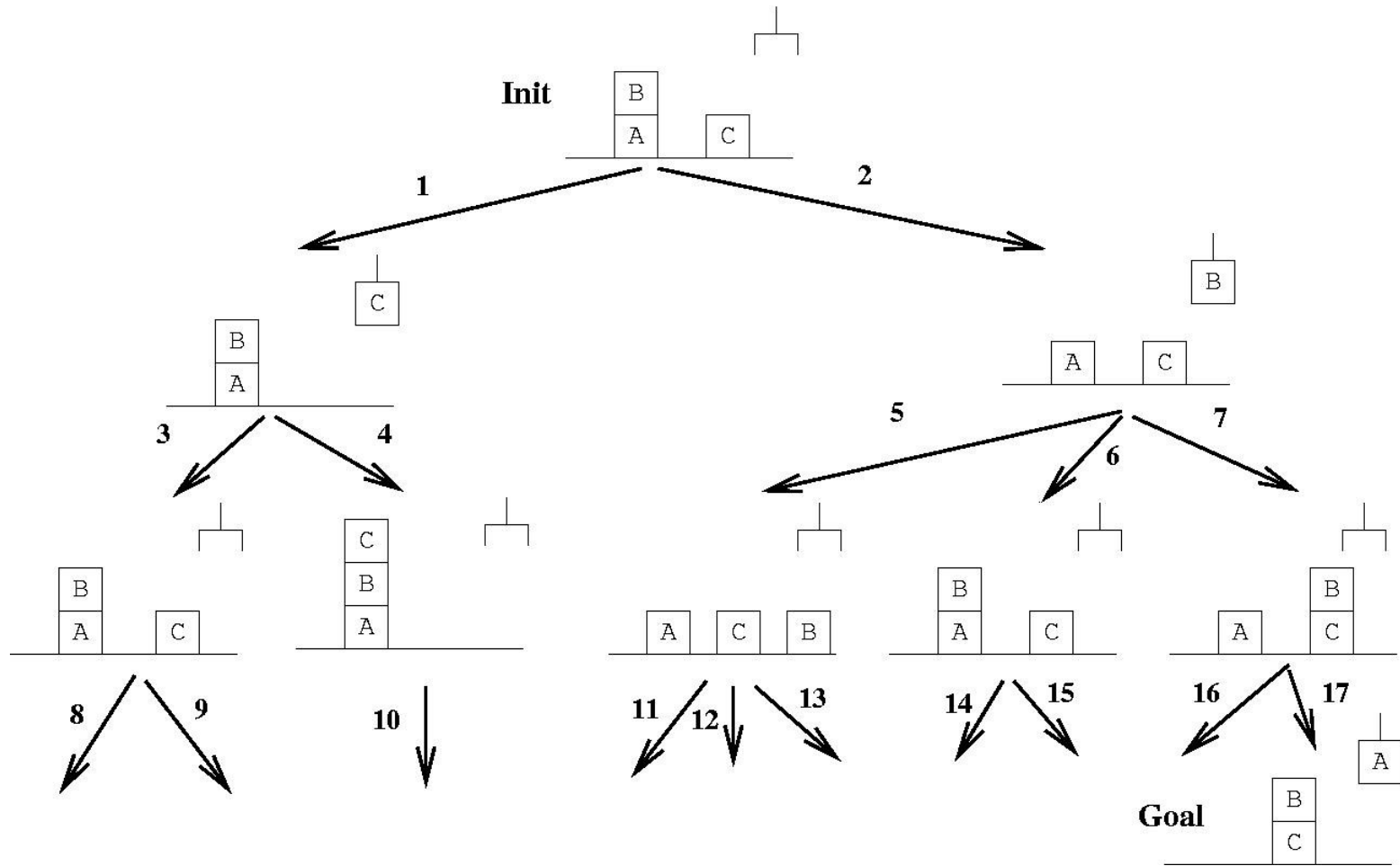
- Planning problems can be solved with **any search algorithms**. Y/N? Explain.
- What is the difference between *Declarative Knowledge* & *Procedural Knowledge*?

Search and planning

- Block world



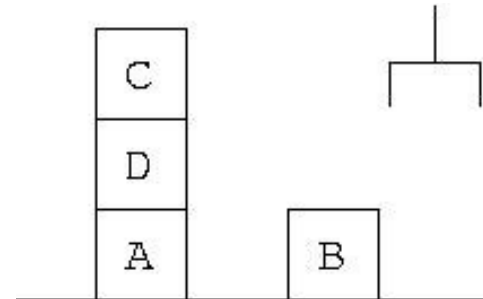
Block world search: BFS



Block world: FOL representation

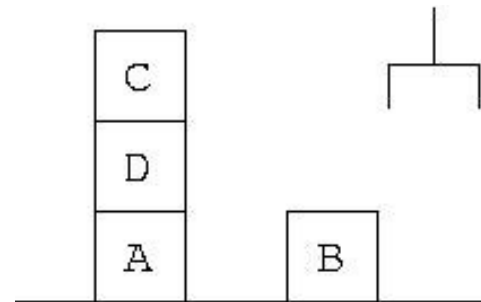
- Block world: Description by predicates of the form
 - **Ontable(x)** x stands on the table
 - **On(x,y)** x stands on y
 - **Clear(x)** nothing stands on x
 - **Handempty** robot hand is empty
 - **Holds(x)** robot hand holds x

- Example
 - **Ontable(?)**
 - ...



Block world: FOL representation

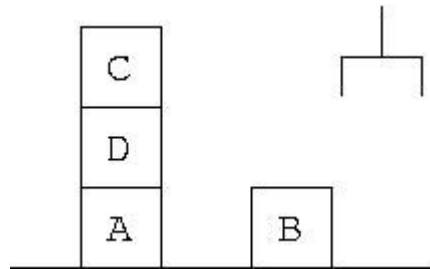
- Block world: Description by predicates of the form
 - **Ontable(x)** x stands on the table
 - **On(x,y)** x stands on y
 - **Clear(x)** nothing stands on x
 - **Handempty** robot hand is empty
 - **Holds(x)** robot hand holds x
- Example
 - Ontable(A)
 - Ontable(B)
 - On(D,A)
 - On(C,D)
 - Clear(C)
 - Clear(B)
 - Handempty



Using First-Order Logic (FOL).

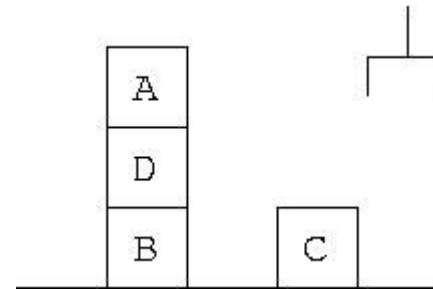
Representation: FOL

- Block world



Initial state

- $\text{Ontable}(A)$
- $\text{Ontable}(B)$
- $\text{On}(D,A)$
- $\text{On}(C,D)$
- $\text{Clear}(C)$
- $\text{Clear}(B)$
- Handempty



Goal state

- $\text{Ontable}(B)$
- $\text{Ontable}(C)$
- $\text{On}(D,B)$
- $\text{On}(A,D)$
- $\text{Clear}(A)$
- $\text{Clear}(C)$
- Handempty



Using First-Order Logic (FOL).

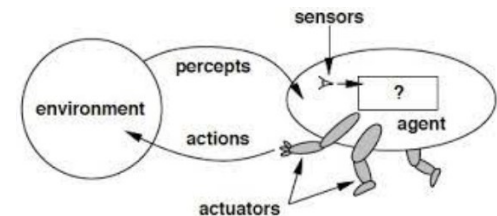
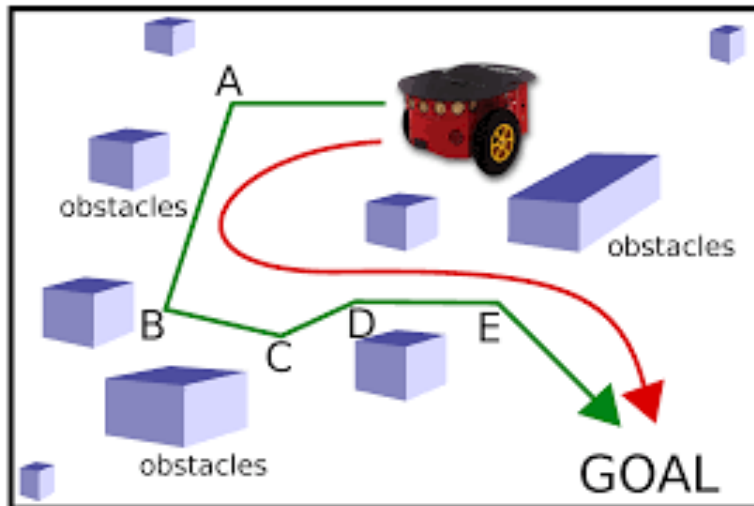
Forward and backward chaining

- Inference in FOL: The approach
 1. decide predicates and functions
 2. construct a knowledge base
 3. search for a **proof**
- Proof by
 - Forward-chaining: starts with known facts and infer new ones until reaching the goal, or
 - Backward-chaining: starts with the goal, and work backwards until hitting known facts

Automated planning

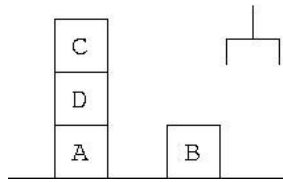
Automated planning

- *Declarative Knowledge* focuses on the description of the world (so far)
- *Procedural Knowledge* focuses on how to change the world (now)
- Need to specify how an action changes the world!



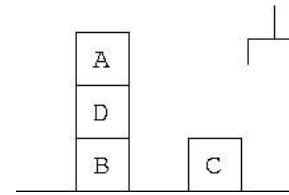
Representation: FOL

- Block world



Initial state

- $\text{Ontable}(A)$
- $\text{Ontable}(B)$
- $\text{On}(D,A)$
- $\text{On}(C,D)$
- $\text{Clear}(C)$
- $\text{Clear}(B)$
- Handempty



Goal state

- $\text{Ontable}(B)$
- $\text{Ontable}(C)$
- $\text{On}(D,B)$
- $\text{On}(A,D)$
- $\text{Clear}(A)$
- $\text{Clear}(C)$
- Handempty

What is missing? **Actions, to change the world.**

Plans and Planning

- *A plan is a sequence of actions* that allows to accomplish a certain task (specified goal).
 - An **action is a primitive operation** that can be carried out by an agent (robot).
- *Planning is the process of determining a sequence of actions* necessary to achieve a specified goal.
 - Planning involves **search**.

Logic Representation of Action

- Form: $A \Rightarrow (B \Leftarrow C)$ ~ action A produces new predicate(s) B when condition(s) C is(are) true.
- Example: Pickup action
 $\forall X (\text{Pickup}(X) \Rightarrow (\text{Gripping}(X) \Leftarrow \text{Gripping}() \wedge \text{Clear}(X)))$
-- For all blocks X, when the gripper is empty and X is clear we can get a new state with the gripper holding X by applying action Pickup

Frame Axioms

- Actions specify what changes.
- Frame problem: all the things that stays the same.
- **Frame axioms are rules** defined to say what predicates describing a state are **not changed** after an operator is applied

Frame Axioms

Performing Unstack

Initial state

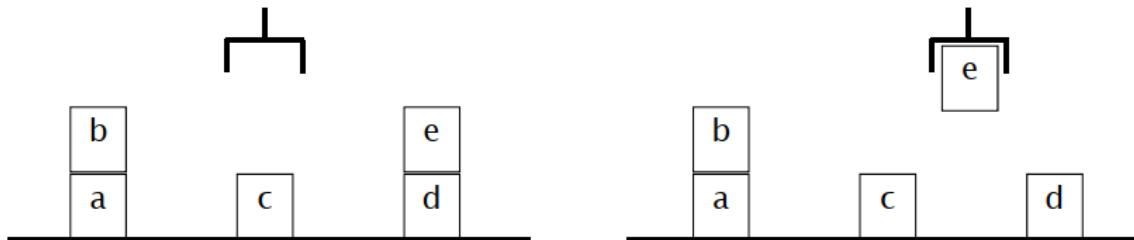
$Ontable(a) \wedge OnTable(c) \wedge OnTable(d) \wedge On(b, a) \wedge On(e, d) \wedge Clear(b)$
 $\wedge Clear(c) \wedge Clear(e) \wedge Gripping()$

Applying action Unstack(e, d)

$\forall X \forall Y (Unstack(X, Y) \Rightarrow ((Gripping(X) \wedge Clear(Y)) \Leftarrow (On(X, Y) \wedge Gripping() \wedge Clear(X))))$

New state after Unstack(e, d) is applied

$Ontable(a) \wedge OnTable(c) \wedge OnTable(d) \wedge On(b, a) \wedge Clear(b)$
 $\wedge Clear(d) \wedge Clear(c) \wedge Gripping(e)$



- Using frame axioms adds exponentially to the search.

STRIPS Representation

- An **action-centric** representation which, for each action, specifies when the action can occur and the effects of the action
- Math: (S, a, g)
 - **S** States: a conjunction of ground and positive literals
 - **a** actions (operators): name parameter list, precondition, and effect (add and delete list)
 - **g** goal: partially specified state

How to Search in STRIPS?

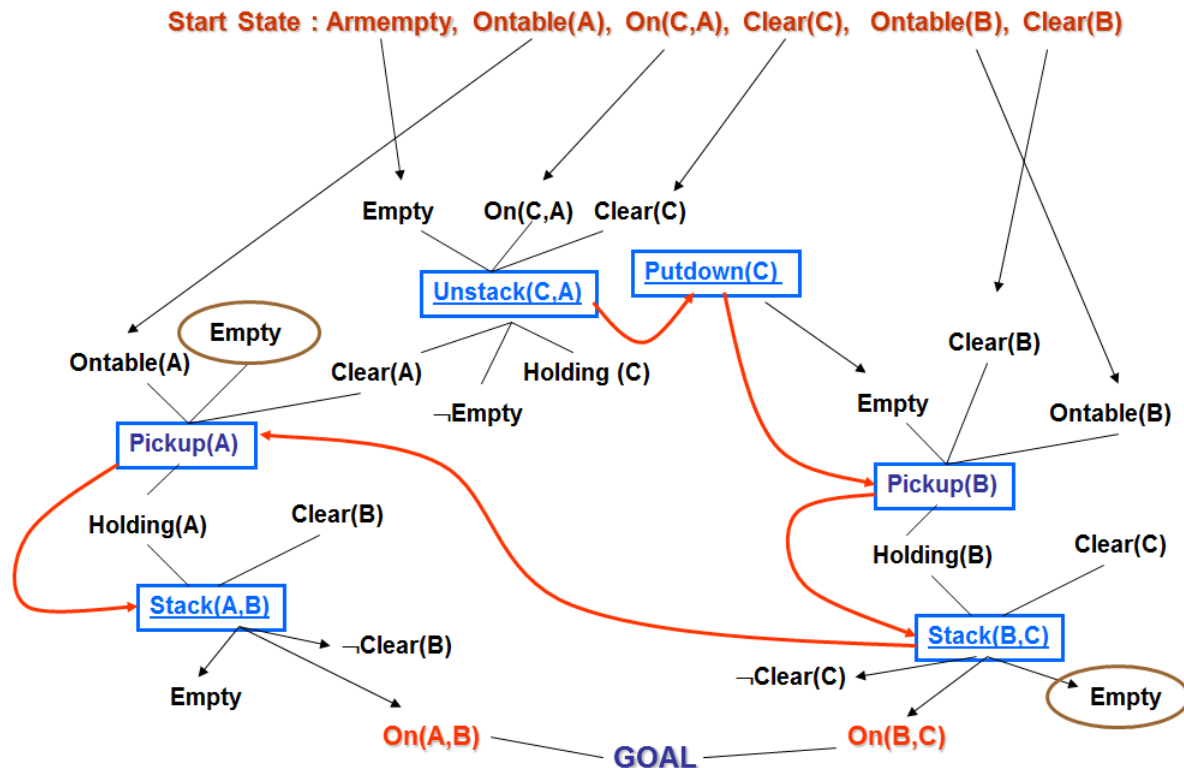
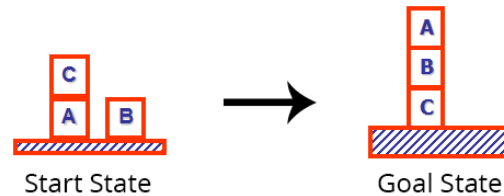
- Simple planning algorithm: BFS, i.e. apply all applicable operators with all possible instantiations one after the other and generate all possible states.
 - intractable
- **Heuristic** in Strips: Search in the search space from the **goal**, apply operators **backward**
- Advantages: Goal often relatively easy compared to initial state

Planning Algorithms

- **State-space search** problem
- Planning problems can be solved with **any search algorithms** while keeping track of actions used to reach the goal.
- Forward state-space search:
 - progression planning
 - irrelevant action problem
 - large state space
- Backward state-space search:
 - regression planning
 - relevant and consistent actions.
- **Heuristic** functions estimate the distance from a state to the goal. Good heuristic functions enables practical feasibility.

Planning as search

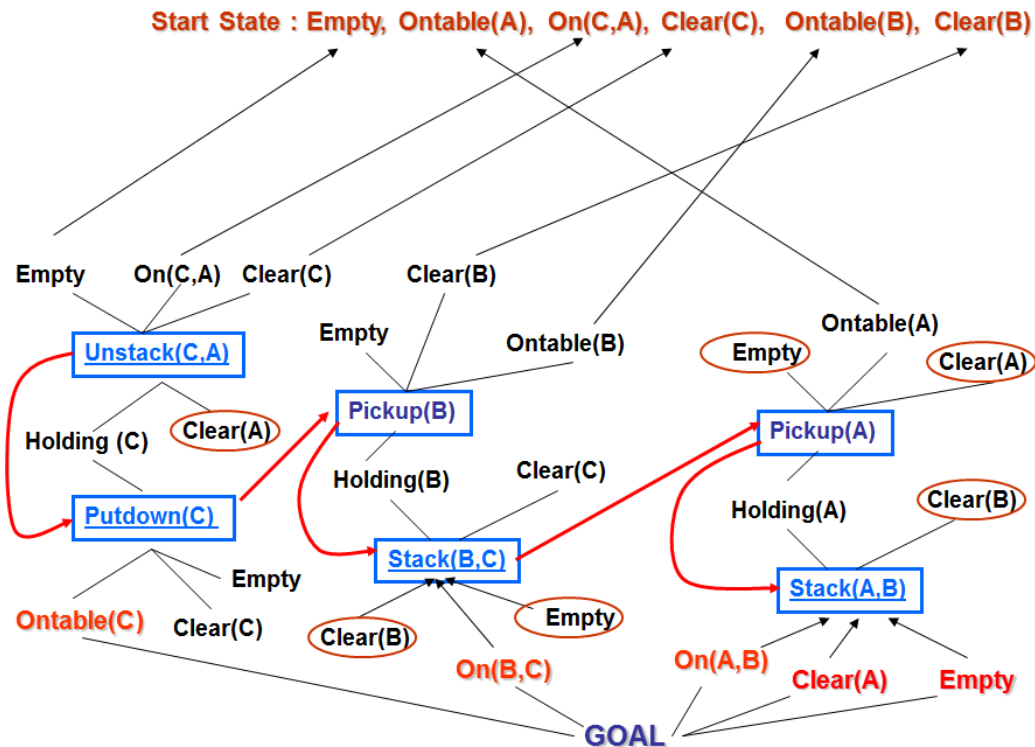
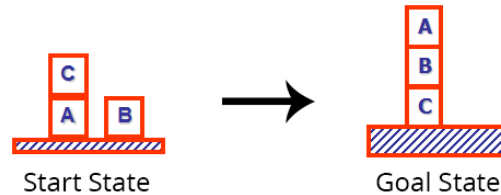
- Forward



Source

Planning as search

- Backward



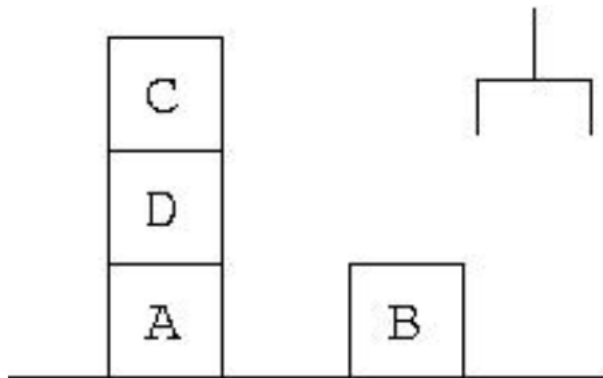
Source

More materials

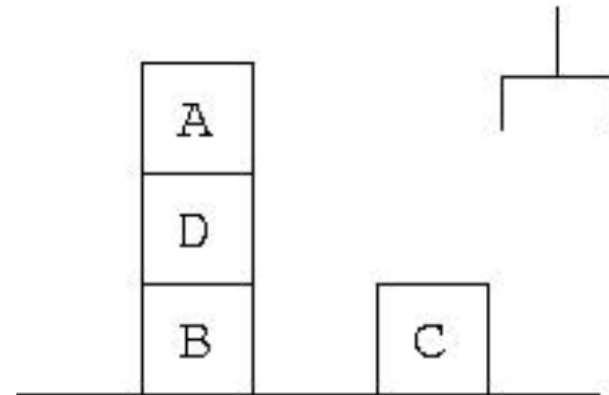
- A* for search for Block world:
<https://github.com/XenoVkl/Blocks-world-problem>
(Python)

STRIPS

- Example: Blocks World

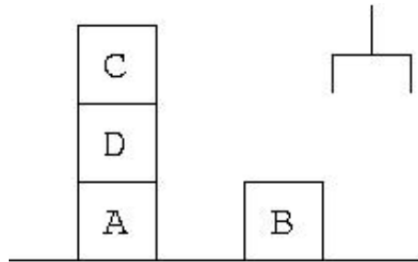


Ontable(A)
 Ontable(B)
 On(D,A)
 On(C,D)
 Clear(C)
 Clear(B)
 Handempty

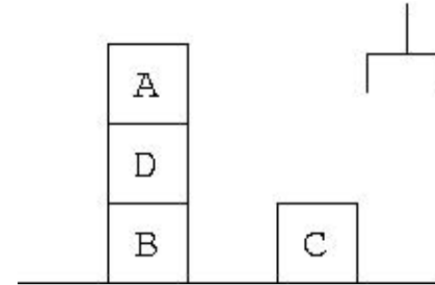


Ontable(B)
 Ontable(C)
 On(D,B)
 On(A,D)
 Clear(A)
 Clear(C)
 Handempty

Example Plan Execution



Ontable(A)
 Ontable(B)
 On(D,A)
 On(C,D)
 Clear(C)
 Clear(B)
 Handempty



Ontable(B)
 Ontable(C)
 On(D,B)
 On(A,D)
 Clear(A)
 Clear(C)
 Handempty

Ontable(A)
 Ontable(B)
 On(D,A)
 On(C,D)
 Clear(C)
 Clear(B)
 Handempty

UNSTACK(C,D)--->

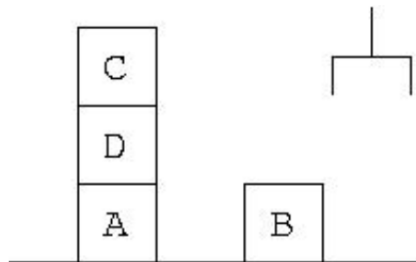
Ontable(A)
 Ontable(B)
 On(D,A)
 On(C,D)
 Clear(C)
 Clear(B)
 Handempty
 Holds(C)
 Clear(D)

PUTDOWN(C)--->

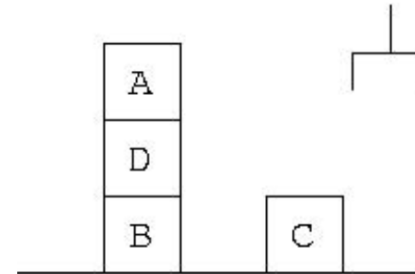
Ontable(A)
 Ontable(B)
 On(D,A)
 Clear(B)
 Holds(C)
 Clear(D)
 Clear(C)
 Ontable(C)
 Handempty

UNSTACK(D,A)--->

Example Plan Execution



Ontable(A)
 Ontable(B)
 On(D,A)
 On(C,D)
 Clear(C)
 Clear(B)
 Handempty



Ontable(B)
 Ontable(C)
 On(D,B)
 On(A,D)
 Clear(A)
 Clear(C)
 Handempty

Ontable(A)

Ontable(A)
 Ontable(B)
 On(D,A)
 On(C,D)
 Clear(C)
 Clear(B)
 Handempty
 UNS
 PUT
 UNSTACK(D,A)---

Ontable(A)
 Ontable(B)
 On(D,A)
 Clear(B)
 Clear(D)
 Clear(C)
 Handempty
 Holds(D)
 Clear(A)
 STACK(D,B)---

Ontable(A)
 Ontable(B)
 Clear(B)
 Clear(C)
 Ontable(C)
 Holds(D)
 Clear(A)
 Clear(D)
 On(D,B)
 Handempty
 PICKUP(A)---

Ontable(A)
 Ontable(B)
 Clear(C)
 Ontable(C)
 Clear(A)
 Clear(D)
 On(D,B)
 Handempty
 Holds(A)
 STACK(A,D)---



```
Ontable(A)
Ontable(B)
On(D,A)
On(C,D)
Clear(C)
Clear(B)
Handempty
```



```
Ontable(B)
Ontable(C)
On(D,B)
On(A,D)
Clear(A)
Clear(C)
Handempty
```

Ontable(A)			
Ont	Ontable(A)		
On(	Onta		
On(	On(	Ontab	Ontable(A)
Cle	On(	Ontab	Ontable(A)
Cle	Clea	On(D, A)	Ontable(B)
Har	Clea	Clear	Clear(B)
	Hand	Clear	Clear(C)
	Hold	Clear	Ontable(C)
UNS	Clea	Ontable	Holds(D)
	Hande	Hold	Clear(A)
		Clear	Clear(D)
		On(D, B)	
		Handempty	
	UNSTA		
	STACK(I		
		PICKUP(A)---	

Ontable(A)
 Ontable(B)
 Clear(C)
 Ontable(C)
~~Clear(A)~~
 Clear(D)
 On(D,B)
 Handempty
 Holds(A)

STACK(A,D)--->

Plan:

```
(UNSTACK(C,D),PUTDOWN(C),UNSTACK(D,A),STACK(D,B),PICKUP(A),STACK(A,D))
```

```

Ontable(B)
Clear(C)
Ontable(C)
Clear(D)
On(D,B)
Holds(A)
Clear(A)
On(A,D)
Handempty

```


STRIPS Representation Example

- Assume that there is a monkey in a room with some bananas hanging out of reach from the ceiling, but a box is available that will enable the monkey to reach the bananas if he climbs on it.
- Initially, the monkey is at A, the bananas at B, and the box at C.
- The monkey and box have height LOW, but if the monkey climbs onto the box, he will have height HIGH, the same as the bananas.
- The actions available to the monkey include GO from one place to another, PUSH an object from one place to another, CLIMB onto an object, and GRASP an object.
- Grasping results in holding the object if the monkey and object are in the same place at the same height.
- The monkey wants to get the bananas.
- Initial State:
 - At(Monkey,A)
 - ...
- Goal State: ?
- Operators:?

Monkey bananas

- Initial State:

At(Monkey,A)

At(Bananas,B)

At(Box,C)

Height(Monkey,Low)

Height(Box,Low)

Height(Bananas,High)

Pushable(Box)

Climbable(Box)

Graspable(Bananas)

- Goal State:

Have(Monkey, Bananas)

Monkey bananas

- Operators:

Go(x,y)

Precond: At(Monkey,x) AND Height(Monkey,Low)

Effect: At(Monkey,y) AND NOT At(Monkey,x)

Push(b,x,y)

Precond: At(Monkey,x) AND Height(Monkey,Low) AND At(b,x) AND Pushable(b) AND Height(b,Low)

Effect: At(b,y) AND At(Monkey,y) AND NOT At(b,x) AND NOT At(Monkey,x)

ClimbUp(b)

Precond: At(Monkey,x) AND Height(Monkey,Low) AND At(b,x) AND Climbable(x) AND Height(b,Low)

Effect: On(Monkey,b) AND NOT Height(Monkey,Low) AND Height(Monkey,High)

Grasp(b)

Precond: At(Monkey,x) AND Height(Monkey,h) AND At(b,x) AND Graspable(b) AND Height(b,h)

Effect: Have(Monkey,b)

Gripper example

- A robot that can move between two rooms and pick up or drop balls with either of his two arms.
- Initially, four balls and the robot are in a first room.
- We want the balls to be in the second room.

Solution:

- Objects: ?
- Predicates: ?
- Initial state?
- Goal state?
- Operators/Actions?

Gripper example

- **Objects:** The two rooms, four balls and two robot arms.
- **Predicates:** Is x a room? Is x a ball? Is ball x inside room y? Is robot arm x empty? ...
- **Initial state:** All balls and the robot are in the first room. All robot arms are empty.
- **Goal specification:** All balls must be in the second room.
- **Actions/Operators:** The robot can move between rooms, pick up a ball or drop a ball

Gripper example

Predicates:

<code>ROOM(<i>x</i>)</code>	– true iff <i>x</i> is a room
<code>BALL(<i>x</i>)</code>	– true iff <i>x</i> is a ball
<code>GRIPPER(<i>x</i>)</code>	– true iff <i>x</i> is a gripper (robot arm)
<code>at-robby(<i>x</i>)</code>	– true iff <i>x</i> is a room and the robot is in <i>x</i>
<code>at-ball(<i>x</i>, <i>y</i>)</code>	– true iff <i>x</i> is a ball, <i>y</i> is a room, and <i>x</i> is in <i>y</i>
<code>free(<i>x</i>)</code>	– true iff <i>x</i> is a gripper and <i>x</i> does not hold a ball
<code>carry(<i>x</i>, <i>y</i>)</code>	– true iff <i>x</i> is a gripper, <i>y</i> is a ball, and <i>x</i> holds <i>y</i>

- In PDDL?

Graph plan

- [RMIT AI - Tute 06 – Graphplan](#)

Hand-on Practice

- AI22 Tut6
- Ask ChatGPT: What is the difference between path planning and automated planning?
 - Any problem with the answer?

4. Tools and Techniques

- Path Planning Techniques:
 - A*, RRT (Rapidly-exploring Random Trees), Dijkstra's algorithm, PRM (Probabilistic Roadmap).
 - Focus on geometric search and optimization.
- Automated Planning Techniques:
 - STRIPS, PDDL, heuristic search (like forward/backward chaining), SAT/Planning Graphs.
 - Focus on symbolic reasoning and action chaining.

Question?